Creative Automation Hub

AI-powered platform for automated creative content generation at scale.

■ Features

Text Generation: Blogs, social posts, ads with AI (Groq/Claude) Image Generation: AI images + template designs Brand Kit: Store colors, fonts, logos for consistency Batch Processing: Generate 100s of variants in parallel Real-time Updates: WebSocket progress tracking Multi-variant: 1-10 text variants, 1-4 image variants per request ■■ Architecture

Hybrid Go + Python + Next.js Stack:

Go Backend (Port 8080): Ultra-fast API, WebSocket, job orchestration **Python Workers**: AI inference (Groq for text, Stable Diffusion for images) **Next.js Frontend** (Port 3000): Modern React UI with real-time updates **Redis**: Job queue for horizontal scaling **PostgreSQL**: Metadata storage **Why Golang?** 10x faster than Python for API requests, excellent concurrency for parallel batch processing, efficient WebSocket handling.

■ Quick Start

Option 1: PowerShell Script (Windows)

```
.\start.ps1
```

Option 2: Docker Compose

```
# Set GROQ_API_KEY in .env
docker-compose up
```

Access: http://localhost:3000

Option 3: Manual

1. Start databases:

```
redis-server
psql -U postgres -c "CREATE DATABASE creative_hub;"
```

2. Backend (Go):

```
cd backend-go
cp .env.example .env
go mod download
go run cmd/server/main.go
```

3. Workers (Python):

```
cd ai-workers
pip install -r requirements.txt
cp .env.example .env
# Add GROQ_API_KEY to .env
python worker.py
```

4. Frontend (Next.js):

```
cd frontend
npm install
cp .env.example .env.local
npm run dev
```

■ Prerequisites

Go 1.23+ Python 3.14+ Node.js 20+ PostgreSQL 18+ Redis 7+ Groq API key (free at https://console.groq.com) ■ Testing

Generate text:

```
curl -X POST http://localhost:8080/api/generate/text \
  -H "Content-Type: application/json" \
  -d '{
    "prompt": "Write a tweet about AI automation",
    "type": "social",
    "tone": "professional",
    "variants": 3
  }'
```

Check status:

```
curl http://localhost:8080/api/jobs/{job_id}
```

Health check:

```
curl http://localhost:8080/health
```

■ Project Structure

```
creative-automation-hub/
■■■ backend-go/           # Go API (Gin framework)
■   ■■■ cmd/server/      # Main entry point
■   ■■■ internal/
■   ■   ■■■ handlers/    # HTTP + WebSocket handlers
■   ■   ■■■ services/    # Job queue, asset management
■   ■   ■■■ models/      # Data structures
■   ■■■ Dockerfile
■■■ ai-workers/          # Python workers
■   ■■■ worker.py        # Main worker loop
■   ■■■ text_generator.py   # Groq/Claude
■   ■■■ image_generator.py  # Stable Diffusion
■   ■■■ Dockerfile
■■■ frontend/            # Next.js 15 + TypeScript
■   ■■■ app/            # App router pages
■   ■■■ components/     # React components
■   ■■■ hooks/          # WebSocket hook
■   ■■■ Dockerfile
■■■ docker-compose.yml
■■■ start.ps1           # Windows startup script
■■■ PROJECT-STATUS.md   # Detailed status
```

■ Environment Setup

backend-go/.env:

```
DB_HOST=localhost
DB_PORT=5432
DB_USER=postgres
DB_PASSWORD=postgres
DB_NAME=creative_hub
REDIS_HOST=localhost
REDIS_PORT=6379
PORT=8080
```

ai-workers/.env:

```
REDIS_HOST=localhost
REDIS_PORT=6379
LLM_PROVIDER=groq
GROQ_API_KEY=your_groq_key_here
```

frontend/.env.local:

```
NEXT_PUBLIC_API_URL=http://localhost:8080
NEXT_PUBLIC_WS_URL=ws://localhost:8080
```

■ Performance

API response: < 50ms WebSocket latency: < 100ms Text generation: 2-5 seconds (Groq) Image generation: 5-15 seconds (Stability AI) Concurrent users: 1000+ Parallel batch jobs: 100+ ■ Documentation

■ *ASGI vs Golang Analysis*

→ **Start here: README-ASGI-COMPARISON.md** - Complete guide to choosing architecture

Quick reference:
- GLOSSARY.md - Acronyms & terms explained (ASGI, JWT, APM, etc.)
- GOLANG-FRAMEWORK-COMPARISON.md - **Why Gin over Fiber, Echo, Chi ■**

Deep Dive:
- ARCHITECTURE-DECISION-MATRIX.md - Decision tree & use cases
- ASGI-VS-GOLANG.md - Complete comparison (10x performance proof)
- CONCURRENCY-MODELS.md - How async/await vs goroutines work
- GOLANG-ADVANTAGES.md - Real benchmarks & cost analysis
- PRODUCTION-CONCERNS.md - Security, monitoring, auth, observability

Architecture & Design

ARCHITECTURE.md - System design overview MVP-DESIGN.md - Feature scope & timeline Setup & Status

SETUP.md - Step-by-step installation PROJECT-STATUS.md - Current implementation status SUMMARY.md - Build summary & file list ■ Deployment

See `docker-compose.yml` for containerized deployment.

For production: Add authentication, S3 storage, monitoring, rate limiting.

■ License

MIT